



CC65 Programación concurrente y distribuida

Enunciado del Trabajo Final del Curso

Profesor:	Carlos Alberto Jara García
Sección:	Todas
Fecha de evaluación:	Semana 15
Ciclo académico:	2025-02

Introducción:

El aprendizaje requiere un estímulo inicial que resulte interesante y nuevo. Precisamente el realizar un análisis del momento actual de la computación, resaltando la evolución de unidades de cómputo en dispositivos con gran capacidad para procesamiento paralelo y conectividad a redes. Así como, buscar la investigación por parte del estudiante de herramientas idóneas para el desarrollo de soluciones y el uso eficaz de los recursos computacionales.

En congruencia con ello, el trabajo final que se propone consiste en la construcción de una solución que implemente una aplicación de programación distribuida y Machine Learning con soporte de Apis, para el problema planteado. El equipo de trabajo utilizará Github como herramienta colaborativa para la gestión del desarrollo de software y Docker compose para el despliegue de la solución.

Objetivo:

El presente documento define el trabajo final y la rúbrica que permite evaluar el logro del curso CC65 Programación Concurrente y Distribuida. El objetivo del trabajo final (TF) es que los estudiantes construyan aplicaciones concurrentes y distribuidas de alto rendimiento de manera eficaz desde el punto de vista de la ingeniería de software.

Logro del curso:

Al finalizar el curso, el estudiante construye aplicaciones concurrentes y distribuidas de alto rendimiento de manera eficaz.

El curso busca desarrollar la competencia general de Razonamiento Cuantitativo en nivel 3 y las competencias específicas Responsabilidad y ética en nivel 2 para Ciencias de computación e Ingeniería de Software.

Instrucciones:

- El trabajo es en equipos de mínimo 2 y máximo 3 estudiantes, se recomienda mantener los grupos formados para el trabajo parcial.
- El trabajo está dividido en 3 entregables.
- Se usará software para detección de plagio.
- El proyecto se aloja en Github y debe seguir Git Flow. Repositorio público.
- El trabajo final será desarrollado íntegramente en GO y contenerizado en Docker.
- Utilizará Docker compose para desplegar la solución, UI, Server API, Nodos.

Alcance del trabajo:

1. Objetivo

Desarrollar e implementar un sistema de recomendación distribuido basado en filtrado colaborativo, capaz de procesar reseñas en paralelo y ofrecer recomendaciones personalizadas con bajo tiempo de respuesta y alta escalabilidad.

Para esto se dispone del dataset MovieLens con un gran volumen de registros.

2. Visión de Arquitectura

El sistema implementa un motor de recomendaciones basado en Filtrado Colaborativo Distribuido. El sistema debe de procesar grandes volúmenes de reseñas de usuarios en un entorno distribuido, aplicando algoritmos de aprendizaje automático para detectar similitudes entre usuarios y hacer recomendaciones personalizadas en tiempo real.

La arquitectura combina un backend de procesamiento distribuido, una API RESTful para la exposición de resultados, y un frontend interactivo que permite visualizar recomendaciones, análisis y métricas.

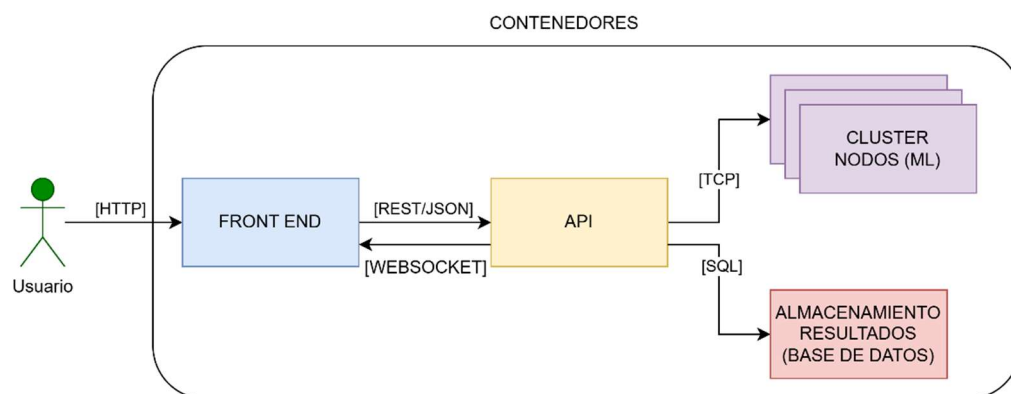


Fig1. Vista de Arquitectura

3. Roles de cada componente

✓ Clúster Nodos (ML):

- Implementa el algoritmo de filtrado colaborativo concurrente/distribuido.
- Procesa grandes volúmenes de datos
- Usa goroutines y channels (Go) para dividir y combinar cálculos.
- Comunica resultados al coordinador (API) mediante TCP interno.

✓ API:

- Implementada en Go (recomendado para continuidad con la lógica concurrente).
- Expone endpoints.
- Puede gestionar la autenticación con JWT tokens.
- Hacer de coordinador del cluster, enviando tareas a los nodos de cómputo y unificando resultados.
- Almacenar resultados de recomendaciones en base de datos.
- Servir datos al frontend en formato JSON usando websockets

✓ Base de datos:

- Incluir dos niveles de almacenamiento:
- Persistente: MongoDB: Usuarios, películas, historial de recomendaciones.
- Cache/colaborativo: Redis: Recomendaciones precalculadas o respuestas parciales.

✓ FrontEnd Web:

- Desarrollar interfaz SPA, use algún framework por ejemplo: Angular, React con componentes simples
- Módulos sugeridos:
 - Inicio de sesión / autenticación.
 - Panel de usuario con sus películas y recomendaciones.
 - Visualización gráfica (p. ej. top películas, similitud entre usuarios).
 - Panel administrador: métricas del cluster (uso CPU, latencia, número de nodos).

4. Metodología

- **Etapas 1: Análisis de Datos**

- Limpieza y selección de campos
- Generación de matriz usuario-película
- Normalización de valores de tiempos o ratings implícitos.

- **Etapas 2: Diseño del Modelo de Filtrado Colaborativo**

- Implementar User-based o Item-based collaborative filtering.
- Calcular similitudes (Cosine Similarity, Pearson Correlation, Jaccard Index).
- Generar recomendaciones a partir de vecinos más cercanos (k-NN).

- **Etapas 3: Paralelización del Cálculo**

- Dividir la matriz en submatrices procesadas por goroutines.
- Utilizar channels para comunicación y sincronización.
- Utilizar patrones de concurrencia
- Medir speedup y scalability del procesamiento.

- **Etapas 4: Distribuido**
 - Implementar comunicación entre nodos (SOCKETS TCP).
 - Desarrollar un coordinador que asigne tareas a nodos.
 - Combinar resultados parciales de similitud y recomendaciones.
- **Etapas 5: Evaluación Experimental**
 - Métricas:
 - Rendimiento: tiempo total de cálculo, uso de CPU, uso de RAM.
 - Escalabilidad: impacto de aumentar cantidad de nodos.
 - Escenarios:
 - Concurrente (multi-worker)
 - Distribuido (multi-nodo).
- **Etapas 6: Integración**
 - Implementar API
 - Implementación de bases de datos
 - Implementar Frontend Web
 - Integrar con el cluster de nodos ML
- **Etapas 7: Documentación y Presentación**
 - Resultados, análisis y conclusiones.
 - Comparativas gráficas y tablas de rendimiento.
 - Propuestas de mejora.

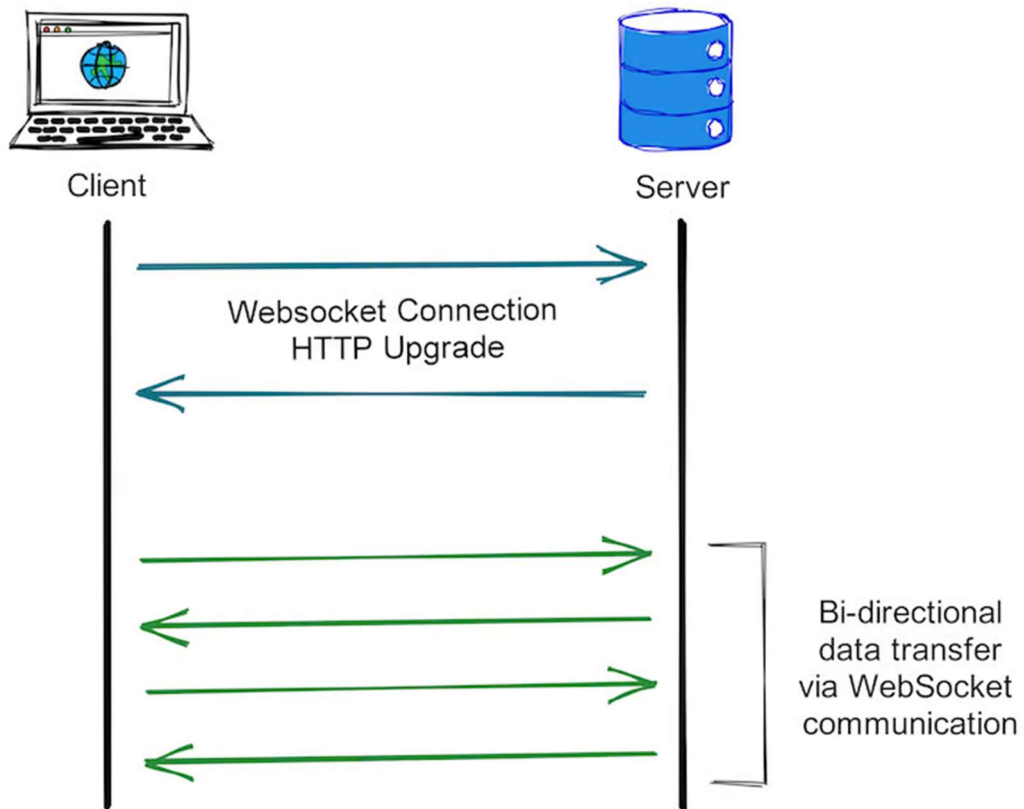
5. Datasets

- MovieLens (10M, 20M, y 25M datasets):
<https://grouplens.org/datasets/movielens/>

6. Anexo

La especificación del protocolo WebSocket define dos esquemas URI:

- WebSocket (ws): used for non-encrypted connections
- WebSocket Secure (wss): used for encrypted connections



Evaluación del Trabajo Final

Instrucciones de Entrega:

- Código fuente de la solución eliminando cualquier defecto.
- Incluir las imágenes y demás recursos utilizados para la elaboración de la solución.
- Video de presentación de la solución en funcionamiento (end to end), realizado en un máximo de 6 minutos donde participen todos los integrantes y demuestren el conocimiento del tema.
- El trabajo será entregado por cada integrante del grupo y mediante el Aula Virtual.
- El plazo es impostergable y por ningún motivo y/o circunstancia se recibirá trabajos fuera de esa fecha y hora, ni por otro medio diferente al aula virtual.

Plazos de Entrega:

- Entregable 1(PC3) – Fecha de entrega semana 11 (08/11/2025 23:30 Hrs.)
 - Elaborar informe de los puntos:
 - Presentación del caso a resolver (Problema y motivación)
 - Análisis de datos
 - Diseño del modelo de filtrado colaborativo
 - Paralelización del cálculo
 - Evidencias de la implementación
- Entregable 2(PC4) – Fecha de entrega semana 13 (22/11/2025 23:30 Hrs.)
 - Elaborar informe de los puntos:
 - Distribución (Cluster de nodos ML)
 - Desarrollo de API
 - Implementación de Bases de datos
 - Evidencias de la implementación y pruebas de funcionamiento
 - Documentación (el informe debe contener el entregable 3)
- Entregable 3(TB2) – Fecha de entrega semana 15 (04/12/2025 23:30 Hrs.)
 - Elaborar informe de los puntos:
 - Desarrollo FrontEnd Web
 - Evaluación experimental
 - Documentación y Presentación Final (el informe final contiene los entregables 3 y 4)

Detalle de Entrega:

1. El archivo ZIP o RAR a presentar tendrá por nombre `YYY_XXXXXXXXXX`, en donde los caracteres `YYY` reemplaza el código de entregable (PC3, PC4, TB2), las `X` se reemplazarán por el código de alumno.
2. Solo se calificarán los trabajos entregados mediante el Aula virtual.
3. Estamos seguros de que cada uno realizará su trabajo, sin embargo, para evitar cualquier perspicacia, [le recomendamos leer el reglamento de disciplina del alumno, en el cual se indican las faltas y las sanciones que se indican en el caso de haber copia de trabajos.](#)

El informe Final debe incluir los siguientes puntos:

- a) Carátula
- b) Resumen
- c) Índice
- d) Descripción del problema y motivación
- e) Objetivos
- f) Desarrollo de la metodología (Evidencias de la implementación de cada etapa)
- g) Conclusiones
- h) Recomendaciones
- i) Bibliografía (utilizar APA7)
- j) Anexos (Link de repositorio GitHub, link de video, documentos, informes, otros)

Links de ayuda

Proyectos en GitHub:

- https://youtu.be/Vjf_s7TGmqY

Rúbrica

	Sobresaliente	En Proceso	Deficiente
Planteamiento	3 puntos Usa diagrama de arquitectura solución detallada para complementar la explicación de su planteamiento solución. Programación concurrente y distribuida. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	2 puntos Define procesos e Información incompleta. Expone y no demuestra conocimiento total del tema.	0 puntos No elaborado
Implementación	8 puntos Funciona correctamente, utiliza docker, puertos, para	5 puntos Elabora el código y ejecuta concurrentemente, usa puertos, canales,	0 puntos No elaborado

	procesar los datos generados por la solución concurrentemente, usa canales y muestra resultados siguiendo las consideraciones de la arquitectura solución. Usa Apis para exponer métodos que consuma el front end y recopile los resultados del cluster. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	muestra resultados, implementando las consideraciones del trabajo de forma parcial. Expone y demuestra conocimiento parcial del tema.	
Interfaz	5 puntos La interfaz en modo gráfico usando un framework UI, muestra un menú de opciones, configura el aplicativo con parámetros ingresados desde la UI, muestra los resultados en tiempo real, demuestra una adecuada experiencia de usuario y organizada en página SPA. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	3 puntos Muestra el funcionamiento de la aplicación y resultados entendible en página web parcialmente. Expone y demuestra conocimiento parcial del tema.	0 puntos No elaborado
Informe	2 puntos Hace buen uso del medio escrito, cumple con la estructura del informe, guarda	1 puntos Sigue una estructura clara y contiene los elementos mínimos necesarios solicitados. Usa	0 puntos No elaborado, No demuestra conocimiento del tema.

	coherencia y presenta resultados, conclusiones entendibles y orientadas a resultados. Anexa el historial del versionamiento del código fuente, evidencia el git Flow. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	herramienta de versionado de código. Informe incompleto. Expone y demuestra conocimiento parcial del tema.	
Video	2 puntos Elabora un video de 6 minutos como máximo donde el grupo presenta el desarrollo y funcionamiento del aplicativo, demostrando dominio del tema.	1 puntos Cubre los puntos solicitados de forma parcial.	0 puntos No elaborar el video afecta la calificación en los demás ítems de la rúbrica.

Santiago de Surco, octubre de 2025